

Lecture 1 - Software Dev Processes

Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software

Software crisis 2.0

- Push factors
 - Hardware advances + cost reduction
 - Data availability + device proliferation
 - Increases in processing power
- Pull factors
 - Digital natives (consuming technology)
- Responses: R&D in
 - Parallel processing
 - Cloud computing
 - Data analytics
 - Security
 - IoT
 - Edge computing
 - Devops, CI, CD
 - Microservices

Types of applications

- Computation & response
 - Real-time
 - Concurrent
 - Distributed
- Nature of code & data
 - Closed-source
 - Open-source
 - Open-content
- Deployment mode
 - Desktop
 - Embedded & edge
 - Cloud-native

Cloud Computing Applications

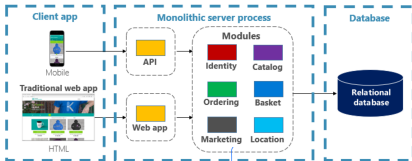
On-site	IaaS	PaaS	SaaS
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	Middleware	Middleware	Middleware
O/S	O/S	O/S	O/S
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking
You manage	You manage	You manage	You manage
Service provider manages	Service provider manages	Service provider manages	Service provider manages

- Software infrastructure hosted on an external data center
 - Software / services delivered over the internet
 - Different models - IaaS (infrastructure), PaaS (platform), SaaS (software)
- Comprises resources, infrastructure, and tools hosted by vendors
- Cloud-enabled: legacy enterprise applications designed for local datacentres, but modified to run on the cloud
 - Retain most features as the legacy system

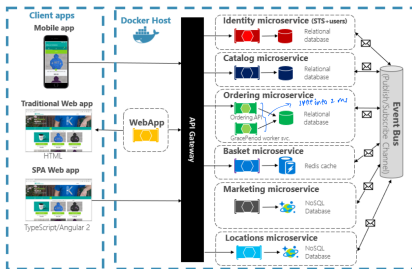
Cloud-native applications

- Cloud native is the software approach of building, deploying, and managing modern applications in cloud computing environments.
- Characteristic features of cloud-native applications:
 - Immutable infrastructure
 - Microservices-based applications
 - API driven
 - Service mesh
 - Containers
 - Dynamically managed
- Cloud-native development: CI/CD, DevOps, Serverless computing

An example of a monolithic web-app design



An example of a cloud-native app design



Cloud-native application stack

- Infrastructure layer
- Provisioning layer
- Runtime layer
- Orchestration and management layer
- Application definition and development layer
- Observability and analysis tools

Software delivery and deployment

Activities that make the software available for use after the development

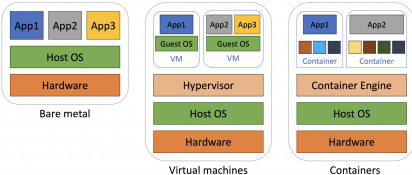
Deployment: process between acquisition of software and execution of software

Deployment decisions can affect the quality attributes

Deployment considerations

- Integration of the internet and related advances
 - Virtual worldwide marketplace
 - Considerations: shift towards cloud-nativity
 - Quality attribute: Portability
- Large-scale content delivery
 - Transfer data/information between various end-points
 - Considerations: location, number of end-points, quality of service
 - Quality attribute: Availability, performance
- Heterogeneous platforms
 - Variety of OS/Browser/Platform combinations
 - Considerations: Coexistence of platforms and interoperability between platforms
 - Quality attribute: Interoperability
- Dependency and change management
 - Multiple components have interdependencies
 - * "uses" relationship
 - Considerations: configurations, version changes (within and outside components), compatibility
 - Quality attribute: Maintainability
- Coordination & communication among components
 - Computations among components need coordination
 - Considerations: API design, nature (synchronous/async) of communication
 - Quality attribute: Performance
- Security
 - Requirements of privacy, authentication, authorization, and integrity
 - Considerations: mechanisms for encryption, authentication & authorization management, attribution
 - Quality attribute: Security, usability

Deployment mechanisms



Bare metal

- From code to executable
 - Catering to target platforms
 - Customized build & linking
 - Factors affecting: availability of libraries & dependencies
- Potentially wasted hardware resources
- Cost factor
- Scalability issues
- Developer productivity reduced
- Benefits:
 - Complete control
 - Physical isolation
- Virtual machines
 - VMs improve resource utilization → reduced cost
 - Flexible, scalable
- Problems:
 - Still a full OS running inside VMs
 - Susceptible to side-channel attacks
 - "Noisy neighbor" problem (if 1 VM uses too much memory, it will affect other VMs)

Containers

- Benefits:
 - Lighter than VMs
 - Include only OS processes and dependencies necessary
 - Better utilization of hardware capacity
 - Allows rapid deployment
 - Write once run anywhere
 - Provision containers to run the applications
 - Ideal for CI/CD, Agile/DevOps practices
 - Granular & controllable
 - Deployments can be of the whole system or of elements within a system
 - Monitor deployment, rollback / patch / redeploy easily
 - Easy integration of the internet and related advances
 - Build cloud-native applications
 - Include runtime with code
 - Caters to heterogeneous platforms
 - Can achieve interoperability & portability
 - Supports dependency and change management
 - Improved maintainability of the software
 - Environment management
 - Applies to both development & deployment
 - E.g., Dockerfile provides the environment description
 - Reproducible
 - Guaranteed to be identical on any system that can run containers
 - Isolation & security
 - Avoid conflicting dependencies
 - Can provide some sand-boxing for code execution
 - Quick to launch (order of milliseconds)
 - Supports DevOps best practices
 - Can be used with orchestrator like Kubernetes
 - Orchestration automates provisioning, deployment, networking, scaling, availability, and lifecycle management of containers
 - Decides how and where containers run
 - Problems:
 - Not suitable for all applications
 - Not suitable for performance-critical applications

Container vs Orchestrator

Containers provide the platform for building and distributing services

- Not suited for running complex applications on its own
- Often need multiple containers, each (set) doing specific tasks / providing specific services

An **orchestrator** is a separate software that:

- Integrates and coordinates many parts (modules, hosted on containers),
- scales up / down the deployment based on demand,
- provides fault tolerance to the application, and
- provides communication among the containers.

Orchestrator works with the containers to realize the production. (e.g. container: docker, orchestrator: kubernetes)

Serverless

- A cloud-native development model
- There are still servers, but developers / enterprises don't have to manage the servers
- Developers package the code in containers

- Application deployed via these containers
- Once deployed, apps respond to demand - automatically scale up / down as needed
- Economics: Metered on demand ⇒ no cost when idle
- Example: AWS Lambda
- Cloud provider manages the physical servers
- Dynamically allocate resources on behalf of developers (or users)
- Code deployed to production
- Works based on Function-As-A-Services (FAAS)
 - Event-driven execution model (run when needed)
 - App logic deployed in containers
 - Such containers are managed by the platform / service provider
- Stateless applications
- Ephemeral - short execution times
- Example applications:
 - Batch processing of image and video
 - Scheduled data compression

Software Delivery

Deployment pipeline is an automated implementation of application build, deploy, test, and release process

CI/CD pipeline is an automated workflow that enables teams to build, test, and deploy code efficiently and reliably

Continuous integration

Build ⇒ Unit Tests ⇒ Deploy to stage ⇒ Acceptance tests

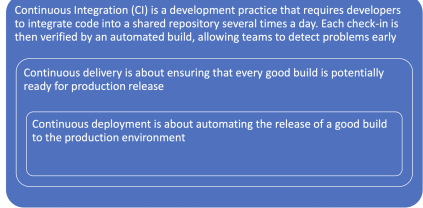
Continuous delivery

Build ⇒ Unit Tests ⇒ Deploy to stage ⇒ Acceptance tests ⇒ (manually) Deploy to production

Continuous deployment

Build ⇒ Unit Tests ⇒ Deploy to stage ⇒ Acceptance tests ⇒ (automatically) Deploy to production

CI/CD Definitions



Continuous delivery and deployment

The ability to get changes of all types (new features, config changes, bug fixes, experiments) into production, or into the hands of users, safely and quickly in a sustainable way.

Benefits:

- Low-risk releases (human error in the process of deploying is reduced, small portions delivered continuously and can get feedback from consumers)
- Faster time to market and early feedback
- Higher quality (continuous feedback → detect bugs earlier and fix earlier)
- Lower cost (saving man-hours due to automation, bugs late in the cycle are expensive)

Continuous delivery: Software can be released into prod at any given time

Continuous deployment: Software is released automatically into prod

Strategies for Continuous Deployment

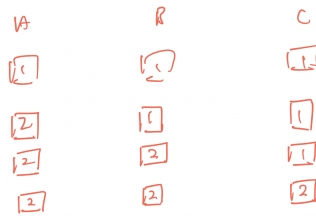
In-place deployment

- Each module/compute is stopped, latest version installed, started, & validated
- Minimal disturbance to the infrastructure
- Reduced infrastructure cost
- Affects availability (bc of downtime when u shut down the old version & bring up the new one)
- E.g., AWS Beanstalk allows one-at-a-time, half-at-a-time, all-at-once mechanisms for deployment

Rolling deployment

- Replace previous versions of the application by completely replacing the infrastructure on which the application is running
- E.g., containers running previous versions of the application will be replaced one-by-one with containers running new versions of the application
- No isolation between old/new versions
- Complicates rollback

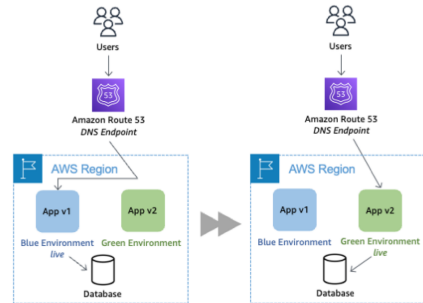
- Some using old version, some using new version



Blue/Green deployment

- Shift traffic between 2 identical environments
- Run different versions of the application in each
- Near-zero downtime
- Allows rollback
- Simplified CI/CD workflow by limiting the complexity

Example: Amazon e-commerce



Canary deployment

- Reduce the risk of deployment
- Incrementally deploy new versions, starting with a small subset of users
- AKA incremental rollout
- Can be used for A/B testing

E.g. Google Chrome Canary

DevOps

DevOps is a set of software development practices that combine software development and operations. The set of practices intends to reduce the time between committing a change to a system, and the change being placed into normal production while ensuring high quality.

DevOps pipeline is a set of automated processes and tools that helps developers and operations professionals to collaborate on building and deploying code to a production environment.

Benefits of DevOps

- Speed of delivery:** The model enables the dev and ops teams to deliver faster for customers. Practices of CI/CD that automate the software release process, from build to deploy can quicker release new features and fix bugs.
- Reliability:** CI/CD ensure testing that each change is functional and safe. Monitoring and logging help stay informed of performance in real-time. These contribute to more reliable delivery and positive experience of end-users.
- Scale:** Operating and managing infrastructure and development processes with automation and consistency help improve scalability. Infrastructure as code helps manage dev, testing, and prod envs in a repeatable and more efficient manner.
- Improved collaboration:** Dev and ops teams collaborate closely, share responsibilities, and workflows. This reduces inefficiencies and saves time (e.g. reduced hand-over periods between dev and ops)

Lecture 2 - Modern Software Engineering

Lecture 3 - Requirements

Requirements are a set of capabilities and constraints [IEEE]

- A condition or capability needed by a user to solve a problem or achieve an objective

2. A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed documents
3. A documented representation to a condition or capability as in (1) or (2)

Example of capability: system should allow a requester to specify pick-up and drop-off locations. **Example of constraint:** pick-up and drop-off locations should be from a list of on-campus buildings / landmarks.

Requirements Development Phases



Elicitation: Discover requirements (through interview, workshop, document analysis, prototyping)

Analysis: Analyze, decompose, derive, understand, negotiate requirements, identify gaps

Specification: Written and illustrated requirements for comprehension, review, and use

Validation: Confirm correct set of requirements that will enable developers to build a solution

Sources of software requirements

- Documents
 - Descriptions of current or competing products
 - Standards or regulations
 - help desk problem reports
- Interviews
 - Focus groups
 - Use case workshops
 - Product champions as key customer rep
- Questionnaires and marketing surveys
 - Run a pilot first - difficult to ask clear questions
- Event-response tables
 - Identify external stimuli and describe system responses
- Observation - watch users do their jobs
 - Create workflow diagrams
 - "Day in the life" studies
 - Look at what information the user has when performing a particular task
- Prototyping
 - Useful for requirements, design, and implementation

FR vs NFR

- FR** describes something the system should do
- specify a behavior or function
 - e.g. the passenger shall be able to print boarding passes for all flight segments for which she has checked in

NFR describes something not directly related to the functionality of the system

- May describe how well the system works
- Can include other attributes too (e.g. reliability, efficiency, usability)
- E.g. mean time between failure $\geq$ 9000 hrs

External vs Internal Quality Attributes

External:

- Observed when the software is executing
- Impacts user's experience of using the software
- Develops the user's perception of software quality
- E.g. Availability, performance, robustness, safety, security, reliability, deployability, compatibility, installability, usability, interoperability, integrity

Internal:

- Not directly observed when software is executing
- Perceived by the developers and maintainers
- Encompasses aspects of design that may impact external attributes
- E.g. Scalability, efficiency, modifiability, verifiability, portability, maintainability, reusability, testability

Lecture 4 - Software Architecture

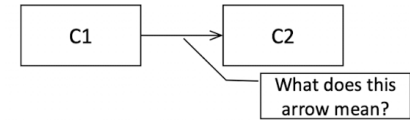
The **Software Architecture** of a program or computing system is the **structure or structures of the system**, which comprise software components, the **externally visible properties** of those components, and the relationships between them.

(Externally visible properties = FR + NFR)

In short, the overall organization of the system and a high level design

Building blocks of architecture

- Component: Element that models an application-specific function, responsibility, requirement, task, or process
- Connector: Element that models interactions among components for the purpose of transfer of control and/or data
- Configuration: Topology or structure



Control flow

- Reasoning is on the computation order (computation in C2 depends on computation in C1)

Data flow

- Reasoning is on the data availability (C2 requires data from C1)

Can use solid / dashed lines to differentiate between the 2 flows and add a legend

Message and Event

- Components communicate via event notifications / message passing / RPC and other protocols
- Async and sync
- Point-to-point

A message is some data sent to a specific address.

- In message-driven systems, each component has a unique address to which other components can send messages
- Each component, or recipient, awaits messages and reacts to them

An event is some data emitted from a component for anyone listening to consume

- An event is emitted by a component when its state changes - record of something that has happened
- They are immutable (cannot be changed or deleted)
- They are ordered in the sequence of their creation
- Sent to the publishing infrastructure, where consumers may later retrieve it

Principle of Modularity

- Modularization $\rightarrow$ shorter development time, better flexibility, better comprehensibility
- Decomposing a big chunk into smaller chunks with well-defined interfaces (APIs)
- A way to manage the complexity of a problem by breaking them down to smaller manageable modules

Horizontal vs vertical slicing

Horizontal slicing — by technical layer

Boundary follows the technology. One slice = one layer, spanning every domain.

	Users	suppliers	Orders	Credits
Presentation	Profile UI	Vendor list UI	Errand board UI	Wallet UI
API & controllers	/users	/vendors	/orders	/credits
Business logic	Auth & roles	Catalogue rules	Lifecycle & expiry	Reserve & transfer
Persistence	users table	vendors table	orders table	credit ledger

Vertical slicing — by business domain

Boundary follows the domain. One slice = one capability, top to bottom.

	Users	Vendors	Orders	Credits
Presentation	Profile UI	Vendor list UI	Errand board UI	Wallet UI
API & controllers	/users	/vendors	/orders	/credits
Business logic	Auth & roles	Catalogue rules	Lifecycle & expiry	Reserve & transfer
Persistence	users table	vendors table	orders table	credit ledger

- Functional
 - Facilities perform only one computation with no side effects
- Layer
 - Related services are kept together, everything else is kept out, and there is a strict hierarchy in which higher-level services can access only lower-level services. Accessing a service may result in side effects
- Communicational
 - Facilities for operating on the same data are kept together
- Sequential
 - A set of procedures, which work in sequence to perform some computation, are kept together. Output is from one input to the next.
- Procedural
 - A set of procedures, which are called one after another, are kept together
- Temporal
 - Procedures used in the same general phase of execution, such as initialization or termination, are kept together
- Utility
 - Related utilities are kept together, when there is no way to group them using a stronger form of cohesion

Types of coupling

- Content
 - Component modifies internal data of another
- Common / global
 - Using global variables; all modules using the global variable are coupled
- Control
 - One module (A) directly controls the other (B) by passing information on what B should do; e.g., using a "flag" that defines what B should do
- Data
 - One module sharing data with another module; e.g., via passing parameters
- External
 - A dependency exists to elements outside the scope of the system, such as the operating system, shared libraries or the hardware.
- Temporal
 - Two actions are bundled together just because they happen to occur at the same time
- Inclusion / import
 - Including a file or importing a package

Architecture styles

- How is the code divided?
 - Technical concern (horizontal)
 - Focuses on the **separation of concerns**
 - May align with the team's expertise (e.g. frontend, backend, db administrator)
 - Domain (business concern) (vertical)
 - Aligned with the domain (business concern)
 - Logical components map to the problem (e.g. customer, payment, shipping)
- How is the system deployed?
 - Single deployed unit (monolithic)
 - Deploy all logical components that make up the application as one unit
 - Application runs as one process
 - Distributed units
 - Application consists of independent logical components
 - Logical components run as individual processes
 - Communicate over the network

		Code partitioning	
		Technical	Domain
Code deployment	Monolith	E.g., Layered	E.g., Modular Monolith
	Distributed	E.g., Event-driven	E.g., Microservices

Types of cohesion